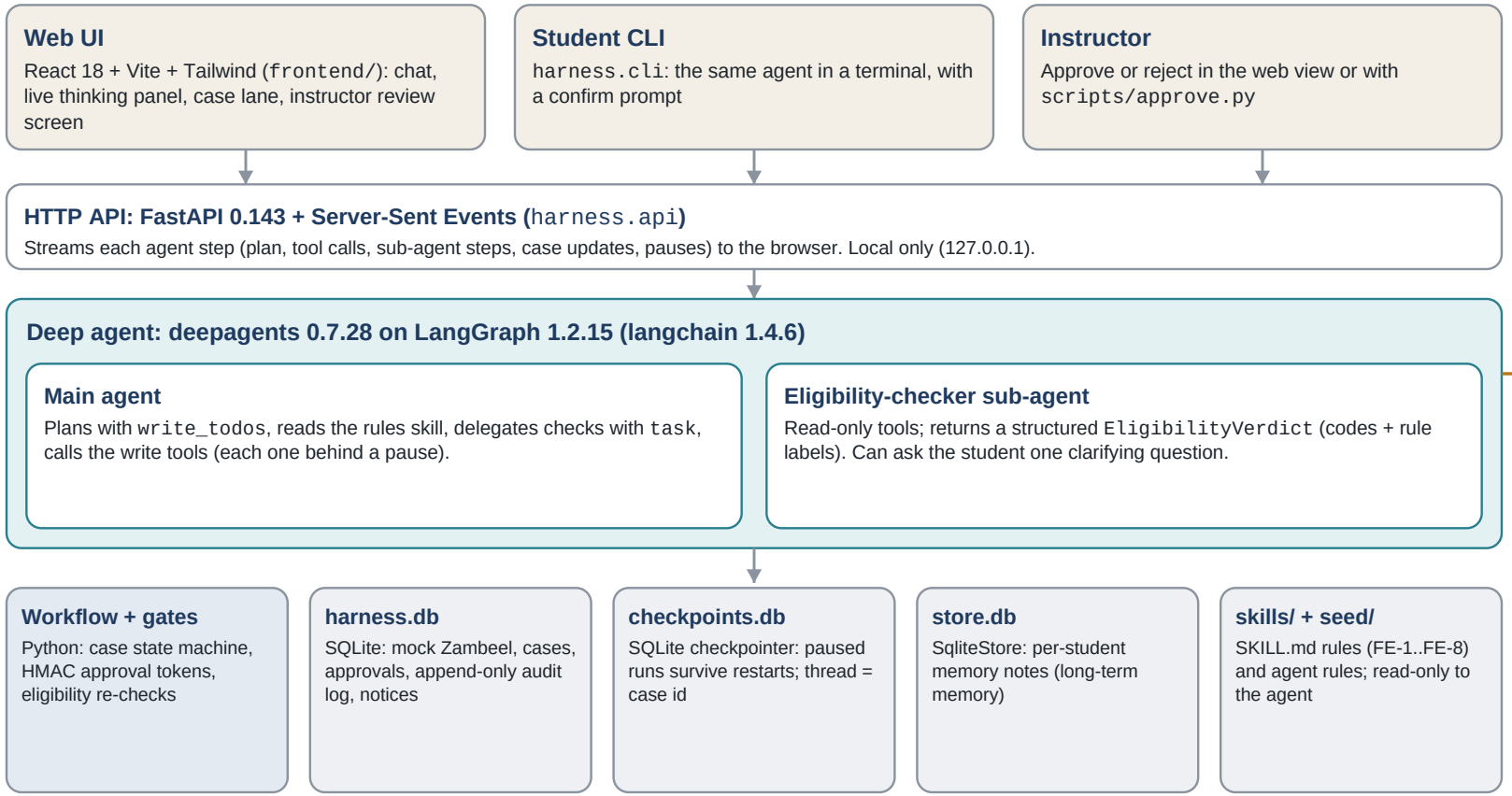
LUMS Process Orchestrator: infrastructure

A deep agent in front of a mock Zambeel. A student asks in plain English; Python enforces every rule.



Outside services

Databricks Free Edition: Model Serving
The LLM: **gpt-oss-120b**, called through its OpenAI-compatible Chat Completions API with langchain-openai `ChatOpenAI`, by the deep agent over HTTPS. Model name set by env var, never hardcoded.

LangSmith
Tracing: every plan, tool call, sub-agent run and pause, for debugging and demos.

GitHub
Private repository `SJK159/sproj_harness`.

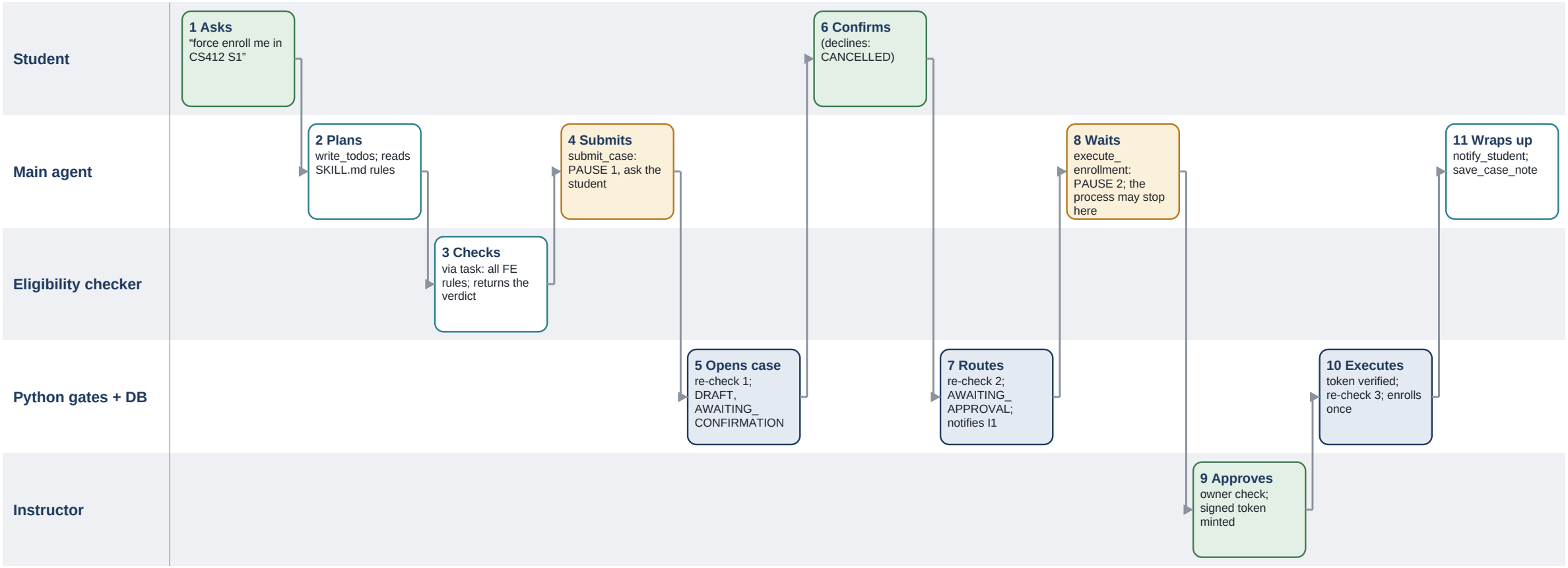
Claude Design
Generated the React frontend from our written design prompt (`docs/frontend-design-prompt.md`).

Google Fonts
Web fonts loaded by the UI (its only external call).

Stack: Python 3.12, uv, pytest · 218 offline tests + 11 live tests on the real model · no real Zambeel / PeopleSoft and no real student data: everything is a synthetic SQLite mock.

One request, end to end

“Can I get force enrolled in CS412 section S1?”: two pauses for people, three eligibility checks, one enrollment.



Case states

DRAFT → AWAITING_CONFIRMATION → SUBMITTED → AWAITING_APPROVAL → APPROVED → EXECUTING → COMPLETED

Other endings: ineligible: no case is ever created · student declines: CANCELLED · instructor rejects: REJECTED (never executes) · token refused or FE cap filled after approval: FAILED. **Every state change writes an append-only audit row** (who, what, from, to, when).

Tool calling

The model chooses the next tool call; Python decides what that call is allowed to do.

Main agent tool	What it does	Kind
`write_todos`	Writes the plan for this request (shown as todos in the UI)	planning
`read_file`	Reads the force-enrollment skill and rules (stands in for RAG)	read
`task`	Delegates the eligibility check to the checker sub-agent	delegation
`get_my_profile`	The student's own year, CGPA, credits, courses	read
`get_my_cases`	The student's own cases and their states	read
`list_offerings`	This term's catalogue: seats, full, FE accepted, slots	read
`submit_case`	Opens the case after a fresh eligibility re-check	write · PAUSE: student confirms
`execute_enrollment`	Verifies the signed approval token, re-checks, enrolls once	write · PAUSE: instructor decides
`notify_student`	Sends the outcome notice	write
`save_case_note`	Adds a memory note built from the case record	write (memory)

Eligibility-checker tool	What it does
`check_my_eligibility`	All FE rules for one section; every failing code with its rule label
`list_sections`	Sections of one course: time, seats, FE terms
`scan_my_fe_options`	The same check on every section this term, grouped
`list_offerings`	The term's catalogue
`get_my_profile`	The student's own record
`read_file`	The rules skill
`ask_student`	One clarifying question (PAUSE: the student's answer is the result)
`EligibilityVerdict`	Structured output: per section can_submit, codes, labels, suggestions

How one tool call is handled

Model proposes
tool name + arguments (gpt-oss-120b)

→

Pause if gated
interrupt_on: approve / reject only; never edit or fake a result

→

Python runs it
student from the runtime context, never from the model's arguments

→

Result back
JSON (or {"error": …}) returned to the model

Guard rails: write tools exist only on the main agent (checked by a test and `scripts/list_agent_tools.py`) · deepagents' general-purpose sub-agent is disabled · its built-in file tools (ls, glob, grep, write_file, edit_file, …) cannot write `/skills`, `/rules` or `/memories` · at most 25 model calls per run (main), 10 (checker).

Key decisions

Each one is logged with its reason and the alternatives in docs/decisions.md.

1. deepagents on LangGraph

Planning, sub-agents, human pauses, memory and skills come built in. One main agent plus one eligibility-checker sub-agent; the automatic general-purpose sub-agent is turned off.

2. Security in Python, not in prompts

No enrollment without a valid approval token: HMAC-signed, 24-hour expiry, single use, bound to the case. Only the instructor side can mint one; no agent tool can.

3. Eligibility re-checked three times

When the case opens, when it is submitted and at execution. The sub-agent's verdict is never trusted for a write, so a seat that fills after approval ends FAILED, not over capacity.

4. People approve through pauses

interrupt_on with approve / reject only (no edit, no faked results). Conditional pauses (when) stop a repeated call from waiting for an instructor who never comes.

5. Durable, auditable state

SQLite checkpointer with thread = case id, so a paused case survives a restart. Append-only audit log; enrollment is idempotent (case id is the key).

6. Memory written only from records

Per-student namespace. save_case_note builds each note from database rows, and the file tools cannot write memory, so nothing said in chat reaches a later prompt.

7. A rules file instead of RAG

A hand-written SKILL.md with placeholder rules FE-1 to FE-8; every reason in an answer cites its rule label. RAG is out of scope for now.

8. Model: gpt-oss-120b on Databricks

Free Edition, via ChatOpenAI. Fixes from live runs: declared the 128K window so summarization can trigger; capped calls per run; streamed step by step because Databricks rejects token-streamed replies.

Where we are

Milestones M0 to M9 done: force enrollment end to end in the CLI and the web UI. 218 offline tests pass, plus 11 live tests on the real model; the integrity check finds every enrollment backed by an approval and every case with a complete audit trail.

Open questions for you

1. Keep the deepagents package for the final system, or move to plain LangGraph?
2. Which workflow next (e.g. dropping a course), and does it need instructor approval?